WEEK 4 · LESSON 22 OF 24 · HANDOUT

Gradient Descent: How Models Learn

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=IHZwWFHWa-w — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 5: Neural Networks — https://www.youtube.com/watch?v=mFZazxxCKbw

Your goal in this lesson: Write gradient descent from scratch, see the effect of the learning rate, and explain how the same loop trains real models.

The idea in one sentence

Start somewhere, find which way is downhill (the derivative), take a small step that way, and repeat.

> *new weight = old weight − learning rate × derivative*

The minus sign moves us **against** the slope — downhill.

Gradient descent on one weight

We want the w that makes error(w) = $(w - 3)^2 + 2$ smallest. We know the answer is 3; let the computer find it. The derivative is $2(w - 3)$.

```
def error(w):
    return (w - 3) ** 2 + 2

def slope(w):
    return 2 * (w - 3)

w = 0.0
learning_rate = 0.1
for step in range(30):
    w = w - learning_rate * slope(w)
    if step % 5 == 0:
        print(step, round(w, 4), round(error(w), 4))
print("final w:", round(w, 4))
```

w moves from 0 towards 3, quickly at first (the slope is steep), then slowly (the slope gets flatter near the bottom).

The learning rate matters

```
def run(lr, steps=30):
    w = 0.0
    for _ in range(steps):
        w = w - lr * slope(w)
    return w

for lr in [0.01, 0.1, 0.5, 0.9, 1.1]:
    print(lr, round(run(lr), 4))
```

Learning rate	What happens
Too small (0.01)	Very slow — still far from 3 after 30 steps
Good (0.1–0.5)	Reaches 3
Large (0.9)	Jumps from side to side, but still settles
Too large (1.1)	**Diverges** — each step overshoots more and w flies away

Choosing the learning rate is one of the most important decisions when training models.

Watching it learn

```
import matplotlib.pyplot as plt

w, lr, history = 0.0, 0.1, []
for _ in range(30):
    history.append(error(w))
    w -= lr * slope(w)

plt.plot(history, marker=".")
plt.title("Error falls as gradient descent runs")
plt.xlabel("step")
plt.ylabel("error")
plt.show()
```

This curve is called the **learning curve**. When you train any model, you watch a curve like this.

Two weights at once

With several weights, compute the partial derivative for each and update all of them together:

```
def error2(w, b):
    return (w - 3) ** 2 + (b + 1) ** 2

w, b, lr = 0.0, 0.0, 0.1
for _ in range(100):
    dw = 2 * (w - 3)
    db = 2 * (b + 1)
    w, b = w - lr * dw, b - lr * db
print(round(w, 3), round(b, 3))   # close to 3 and -1
```

A neural network does exactly this — for millions of weights at once, using backpropagation to compute every partial derivative.

Variations you will hear about

- **Batch gradient descent** — use all the data for each step (what we will do next lesson).
- **Stochastic gradient descent (SGD)** — use one random example per step: noisy but fast.
- **Mini-batch** — use a small group (e.g. 32 examples) per step: the usual choice.
- **Adam** — a popular method that adapts the step size for each weight automatically.

Local minima

Some error curves have several valleys. Gradient descent finds *a* valley — not always the deepest one. In practice, for large neural networks this is usually not a big problem, but it is worth knowing.

Key terms

- **Gradient descent** — repeatedly step against the gradient to reduce error.
- **Learning rate** — the step size.
- **Step / iteration** — one update of the weights.
- **Divergence** — the error grows instead of shrinking (learning rate too large).
- **Learning curve** — error plotted against training steps.
- **Local minimum** — a valley that is not the lowest overall.

Remember this: Gradient descent is a simple loop — compute the slope, step downhill, repeat. The learning rate controls the step size: too small is slow, too large diverges.

Name: _____ Date: _____

Activity

Task 1

Change the starting point to w = 10. Does gradient descent still reach 3?

Task 2

Find the largest learning rate that still converges for this error function. (Hint: try values between 0.9 and 1.1.)

Task 3

Use gradient descent to find the minimum of $f(x) = x^2 - 6x + 13$. (Slope: $2x - 6$.) Check your answer by hand.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is the gradient descent update rule?
   A. w = derivative
   B. w = w + learning rate × derivative
   C. w = w × 2
   D. w = w − learning rate × derivative
2. What happens when the learning rate is far too large?
   A. Training is slow but safe
   B. The model becomes smaller
   C. Nothing changes
   D. The steps overshoot and the error can grow (divergence)

3. What happens when the learning rate is very small?

A. The data is deleted
B. Divergence
C. Instant convergence
D. Very slow progress

4. With error(w) = $(w - 3)^2$, where does gradient descent end up?

A. w = 3
B. w = -3
C. w = 9
D. w = 0

5. What is a learning curve?

A. A chart of error against training steps
B. A school timetable
C. The shape of the data
D. A derivative

6. What is mini-batch gradient descent?

A. Updating weights using a small group of examples each step
B. Using no data
C. Training only once
D. Using one tiny model

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.